

二、TF 构建实验指导书

1. 实验描述

计算图是 ROS 处理数据的一种点对点的网络形式。程序运行时，所有进程以及他们所进行的数据处理，将会通过一种点对点的网络形式表现出来，这里涉及一些重要的概念，包括节点、话题、消息与服务。

在 ROS 中，TF (Transform) 是一种用于坐标转换的机制。它是一个非常重要的概念，被定义为坐标转换的标准规范，这套标准定义了坐标转换的数据格式和数据结构。

TF 能够随时跟踪多个坐标系，使用树形数据结构来缓冲并维护多个参考系之间的坐标变换关系。这种变换关系可以帮助开发者在任意时间，将点、向量等数据的坐标在两个参考系中完成坐标变换。

通俗来讲，TF 能够计算出伺服机器人每个部件间的相对位置关系以及机器人与世界坐标间的关系，通过这些关系可以用于计算相关的位置和姿态信息，以便于相关功能的实现。

本次实验的目标就是学习话题的发布与接收操作、TF 坐标的相关知识，理解其逻辑及构建、使用方法。

2. 实验目标

- (1) 话题的发布与接收操作
- (2) 学习 TF 坐标变换的基本知识，理解其调用以及坐标系编辑的方法；
- (3) 学习静态/动态坐标系的构建逻辑及方法，并在 RViz 中查看其效果；
- (4) 查看坐标系话题的相关消息以及坐标系间的节点关系图。

3. 实验步骤

步骤 1：话题的发布与接收操作

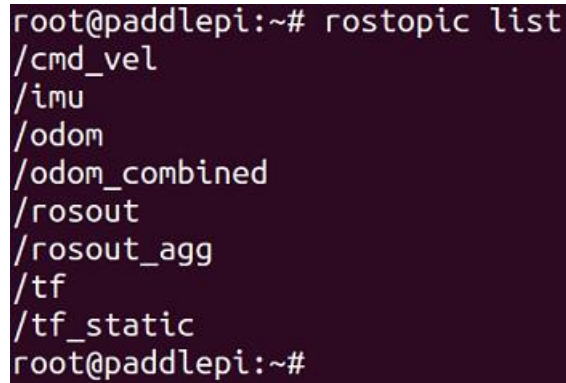
话题是 ROS 中用于消息传递的机制。通过话题可以实现节点间的通信，以此实现节点间的协作。在 ROS 中，执行对应的节点功能后，可以在命令行中输入对应的命令从而查看话题相关的内容。

下面以手柄遥控为例，查看其话题的相关内容。

(1) 查看话题列表及话题内容

运行手柄遥控的 launch 文件后（`roslaunch sebot_robot sebot_joystick.launch`），开启一个新终端，在终端中输入以下命令即可查看当前所有的话题列表。

```
rostopic list
```



```
root@paddlepi:~# rostopic list
/cmd_vel
/imu
/odom
/odom_combined
/rosout
/rosout_agg
/tf
/tf_static
root@paddlepi:~#
```

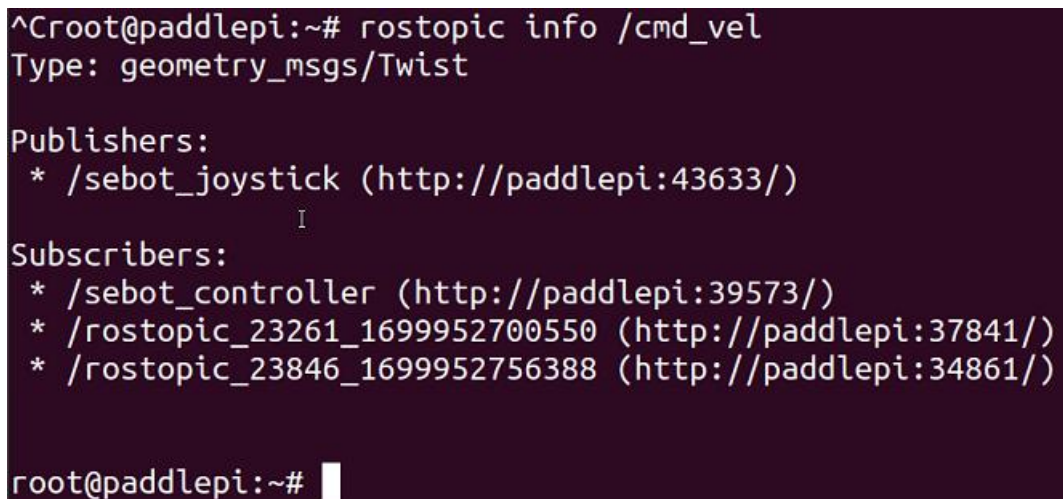
图 1-42 查看话题列表

如上图示，通信中有 `/cmd_vel`、`/imu`、`/odom`、`/tf` 等话题列表。

获得话题列表后，输入以下命令即可查看 `/cmd_vel` 话题的发布者和接收者。

```
rostopic info /cmd_vel
```

Publishers 即发布者，负责向话题发送消息，Subscribers 即订阅者，负责从话题内接收数据。



```
^Croot@paddlepi:~# rostopic info /cmd_vel
Type: geometry_msgs/Twist

Publishers:
* /sebot_joystick (http://paddlepi:43633/)

Subscribers:
* /sebot_controller (http://paddlepi:39573/)
* /rostopic_23261_1699952700550 (http://paddlepi:37841/)
* /rostopic_23846_1699952756388 (http://paddlepi:34861/)

root@paddlepi:~#
```

图 1-43 查看话题发布者和订阅者

通过类似命令可以分别查看 `/imu`、`/odom`、`/tf` 等话题的发布者及订阅者。

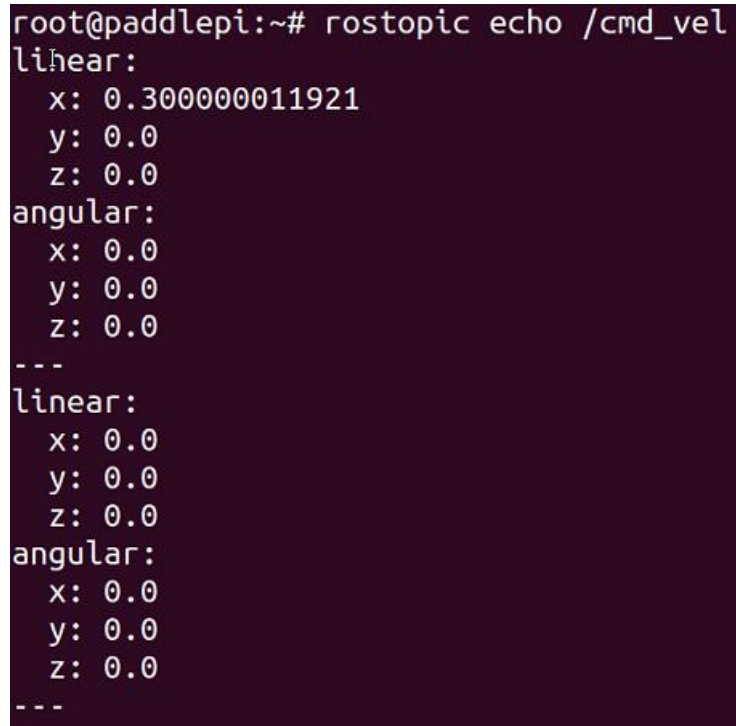
(2) 订阅并接收话题消息

在终端中输入以下命令可以订阅话题/cmd_vel 并显示其内容。

```
rostopic echo /cmd_vel
```

此话题内的数据为按下遥控手柄的信息，因此在按下手柄的按键后，终端才会出现相关的数据，否则是没有数据的。

接收的消息是对机器人底盘的控制，linear 即线速度，angular 即角速度，x、y、z 表示的是三维坐标轴。当机器人发生移动或旋转时，对应的数据会发生变化。

A terminal window with a dark background and light-colored text. The prompt is 'root@paddlepi:~#'. The command 'rostopic echo /cmd_vel' has been entered. The output shows two sets of data. The first set is for 'linear' velocity with x: 0.300000011921, y: 0.0, and z: 0.0. The second set is for 'angular' velocity with x: 0.0, y: 0.0, and z: 0.0. There are three dashes between the two sets of data, and another three dashes at the bottom of the visible output.

```
root@paddlepi:~# rostopic echo /cmd_vel
linear:
  x: 0.300000011921
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

图 1-44 订阅并输出话题信息

(3) 发布话题消息

运行手柄遥控程序时，节点间会自行在话题中发布和接收数据，同时我们也可以通过在终端中输入指令来手动发布话题数据。在终端中输入以下指令即可在/cmd_vel 话题中发布使机器人旋转的控制信息。其中 rostopic pub -r 10 是默认指令，/cmd_vel 是话题名称，geometry_msgs/Twist 是数据类型和格式，linear 即线速度，angular 即角速度，x、y、z 表示的是三维坐标轴。

```
rostopic pub -r 10 /cmd_vel geometry_msgs/Twist '{linear: {x: 0, y: 0, z: 0}, angular: {x: 0, y: 0, z: 0.1}}'
```

指令执行后/cmd_vel 话题内会重复发出对应的消息，在新终端中订阅并接收/cmd_vel 话题消息即可看到消息内容。订阅节点并接收到消息内容后便会控制机器人原地转圈，按下

“ctrl” + “c” 即可停止在此话题发布数据。由于话题数据已经被接收，因此需要覆盖数据才能停止机器人旋转，所以在停止消息发布的终端后，切换到手柄控制的终端并按下手柄上的任意一个方向键并松开即可覆盖数据使机器人停止转动。

```
root@paddlepi:~# rostopic pub -r 10 /cmd_vel geometry_msgs/Twist '{linear: {x: 0,y: 0,z: 0},angular: {x: 0,y: 0,z: 0.1}}'
```

图 1-45 向话题内发布数据

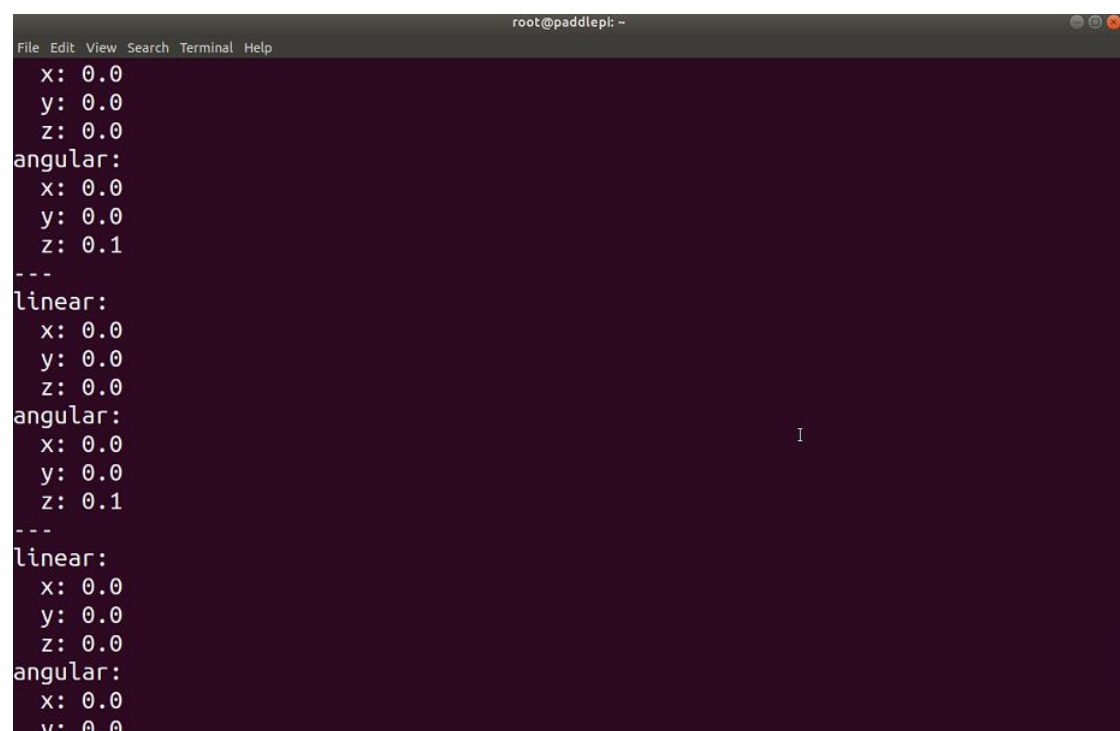


图 1-46/cmd_vel 话题内重复生成消息

ROS 系统中自带一个小海龟仿真器，该仿真器提供了硬件抽象、设备驱动、库函数、可视化、消息传递和软件包管理等诸多功能，运行此仿真器可执行控制及话题的相关操作，比如，通过键盘的方向键可以控制小海龟前后左右移动。

开启一个终端 Terminal，输入以下命令启动单一节点管理器：

```
roscore
```

打开新终端，执行以下命令开启小海龟仿真器：

```
roslaunch turtlesim turtlesim_node
```

再打开新终端，执行以下命令即可通过键盘方向键控制小海龟移动。

```
roslaunch turtlesim turtle_teleop_key
```

小海龟仿真器中也可进行话题的相关操作，具体步骤与步骤 6 相一致，但是需要修改部分内容，例如通过发布话题消息使小海龟旋转时就需要修改话题名称，具体指令为

```
rostopic pub -r 10 turtle1/cmd_vel geometry_msgs/Twist '{linear: {x: 0, y: 0, z: 0}, angular: {x: 0, y: 0, z: 0.1}}'
```

其中将话题名称修改为 turtle1/cmd_vel，接受话题消息时也需要使用此话题名称。

关于小海龟仿真器的更多操作，可以自行查阅相关资料。本手册主要的内容为商用机器人的使用，所使用仿真工具主要是 RViz，在后续将进一步详细介绍。

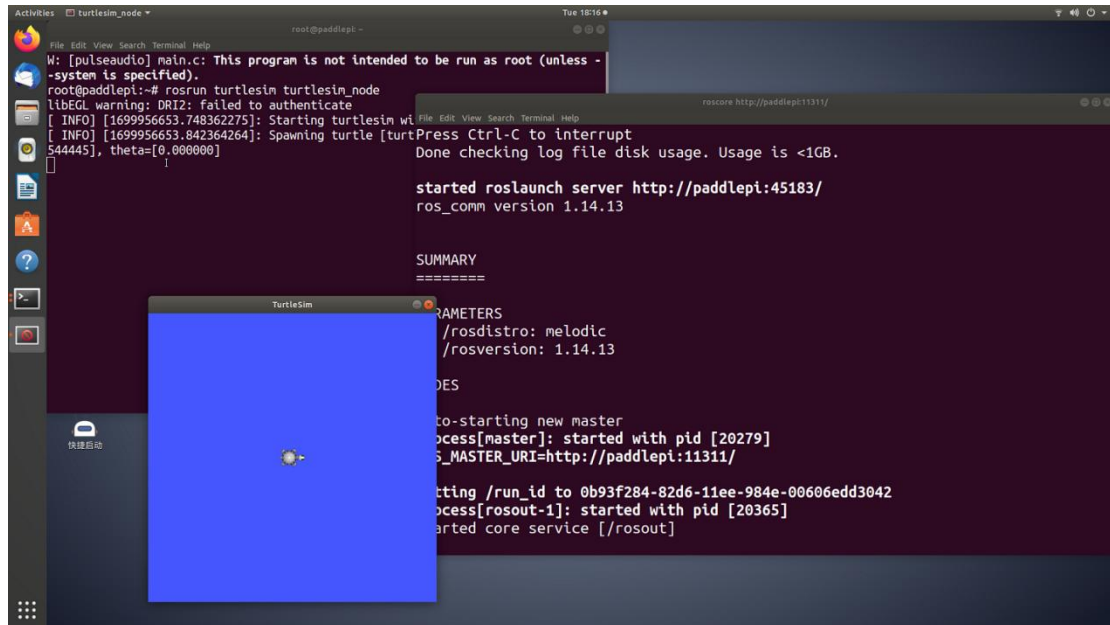


图 1-47 运行小海龟仿真器

步骤 2：TF 基本原理

ROS 中的 TF 坐标是一个用于描述物体在空间中的位置和姿态的功能包。它能够处理不同坐标系之间的相对位姿关系，并且可以随时间跟踪多个参考系。在 ROS 中，TF 坐标主要分为两类：静态 TF 和动态 TF。静态 TF 主要描述的是机器人模型中各个部件之间相对静止的位姿关系，即机器人发生位姿变换时，所有静态坐标均会发生变化；而动态 TF 主要描述的是不同坐标系间的数据随着时间的变化而更新，即机器人的位姿发生变换时，其动态坐标数据也会发生变化，例如机器人在固定的地图上移动时，地图坐标固定不变，机器人在地图中的位置则会随着移动发生改变。

TF 坐标的功能主要包括坐标转换和坐标系转换。坐标转换是一个坐标在不同坐标系下的表示，而坐标系转换则是不同坐标系之间的相对位姿关系。这两种功能可以帮助开发者在不同参考系之间进行坐标变换，从而更好地理解和控制机器人在空间中的运动。例如，如果一个机器人的手臂在不同参考系下进行移动，那么通过 TF 坐标可以将手臂的位置和姿态从一种参考系转换到另一种参考系中，以便于进行后续的控制和规划，其基本原理来自于机器

人学中的运动学建模。此外，TF 坐标还可以用于机器人的感知和感知建图等任务中。例如，通过使用 TF 坐标，可以将从传感器获取的数据（如激光雷达或摄像头的的数据）从传感器坐标系转换到机器人坐标系中，以便于后续的数据处理和分析。

在 TF 变换中，通常使用六个参数来进行坐标变换的计算，六个参数中常包括三个平移参数和三个旋转参数。平移参数指的是子坐标系相对于父坐标系在三维轴 x、y、z 上的偏移量，旋转参数指的是在三个坐标轴上的旋转角度，分别是俯仰角、横滚角以及偏航角，后续实操时我们通过相关参数来验证。

理解其基本原理后我们在后续的步骤中去实际操作商服机器人的 TF 坐标变换功能。

步骤 3：TF 静态坐标发布逻辑

TF 坐标系构建及相关实验也需要通过编程来实现。商服机器人工程文件中有相关源码文件。可以在个人 PC 中通过远程连接方式打开工程文件夹。

（1）源码逻辑

打开工程文件夹，从文件夹 `src\sebot_robot\src` 中找到静态坐标变换的相关源码。

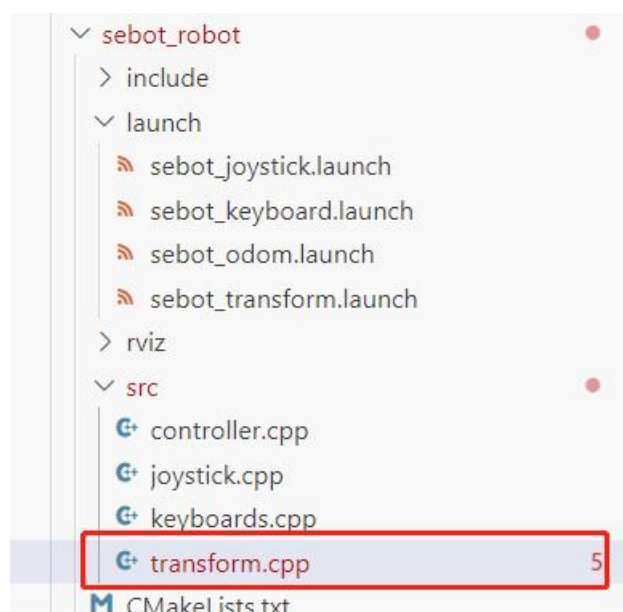


图 2-1 静态坐标变换源码

单击此文件即可查看源码的内容。此源码文件由 c++编写，由于其他功能会调用此源码，因此我们只做原理解，后续实操时在 launch 文件中进行修改。

程序代码不多，大部分功能都有注释，结合注释理解逻辑即可。这里主要讲解程序整体的逻辑。

程序中首先是导入所需的库文件，然后定义了主函数。

主函数中先初始化 `ros` 功能以及坐标系发布者，接着就是将相关模块的静态坐标系进行设置，设置时需要定义六个参数，即三个旋转参数和三个平移参数，注释中的空间向量即旋转参数，表示在三维轴上的旋转角度，三维坐标即当前模块相对于基坐标系在三维轴上的平移距离。基坐标系即机器人底盘的坐标，其他坐标与基坐标间的位姿关系可以自由定义，实际应用中为了满足精准的需求，一般都是确定机器人结构后通过手动测量得出具体的参数的。程序中一共定义了七个静态坐标系，加上基坐标系，因此在 `RViz` 监视器中我们最多可以看到八个坐标系

程序最后是进入到循环中，因为需要重复发送坐标消息以供坐标变换使用。

```
#include <ros/ros.h>
#include <std_msgs/String.h>
#include <tf/transform_broadcaster.h>
#include <sstream>
using namespace std;

int main(int argc, char **argv)
{
    ros::init(argc, argv, "sebot_transform");
    tf::TransformBroadcaster tfBroadcaster; // 坐标系数据发布者

    //[01] 定义IMU静态坐标系
    tf::Transform tfImu;
    tf::Quaternion quater; // 四元数变换
    quater.setRPY(0, 0, 0); // 空间向量 rad q.setRPY(roll, pitch, yaw);
    tfImu.setOrigin(tf::Vector3(0, 0, 0)); // 三维坐标
    tfImu.setRotation(quater);

    //[02] 定义激光雷达静态坐标系
    tf::Transform tfLaser;
    tfLaser.setOrigin(tf::Vector3(0, 0, 0.15)); // 三维坐标: m
    quater.setRPY(0, 0, -3.1416); // 空间向量: rad q.setRPY(roll, pitch, yaw);
    tfLaser.setRotation(quater);

    //[03] 定义深度相机静态坐标系
    tf::Transform tfCamera;
    tfCamera.setOrigin(tf::Vector3(0.05, 0, 1)); // 三维坐标
    quater.setRPY(0, 0, 0); // 空间向量 q.setRPY(roll, pitch, yaw);
    tfCamera.setRotation(quater);

    //[04] 定义超声波静态坐标系
```



```

tf::Transform tfUltraLF;
tfUltraLF.setOrigin(tf::Vector3(0.2, 0.1, 0.07)); // 三维坐标
quater.setRPY(0, 0, 0); // 空间向量 q.setRPY(roll, pitch, yaw);
tfUltraLF.setRotation(quater);

//[05] 定义超声波静态坐标系
tf::Transform tfUltraMF;
tfUltraMF.setOrigin(tf::Vector3(0.2, 0, 0.07)); // 三维坐标
quater.setRPY(0, 0, 0); // 空间向量 q.setRPY(roll, pitch, yaw);
tfUltraMF.setRotation(quater);

//[06] 定义超声波静态坐标系
tf::Transform tfUltraRF;
tfUltraRF.setOrigin(tf::Vector3(0.2, -0.1, 0.07)); // 三维坐标
quater.setRPY(0, 0, 0); // 空间向量 q.setRPY(roll, pitch, yaw);
tfUltraRF.setRotation(quater);

//[07] 定义超声波静态坐标系
tf::Transform tfUltraMB;
tfUltraMB.setOrigin(tf::Vector3(-0.2, 0, 0.07)); // 三维坐标
quater.setRPY(0, 0, -3.1416); // 空间向量 q.setRPY(roll, pitch, yaw);
tfUltraMB.setRotation(quater);

ros::Rate loop_rate(20);
while (ros::ok())
{
    tfBroadcaster.sendTransform(tf::StampedTransform(tfImu,
ros::Time::now(), "base_footprint", "imu"));
    tfBroadcaster.sendTransform(tf::StampedTransform(tfLaser,
ros::Time::now(), "base_footprint", "laser"));
    tfBroadcaster.sendTransform(tf::StampedTransform(tfCamera,
ros::Time::now(), "base_footprint", "camera"));
    tfBroadcaster.sendTransform(tf::StampedTransform(tfUltraLF,
ros::Time::now(), "base_footprint", "ultraLF"));
    tfBroadcaster.sendTransform(tf::StampedTransform(tfUltraMF,
ros::Time::now(), "base_footprint", "ultraMF"));
    tfBroadcaster.sendTransform(tf::StampedTransform(tfUltraRF,
ros::Time::now(), "base_footprint", "ultraRF"));
    tfBroadcaster.sendTransform(tf::StampedTransform(tfUltraMB,
ros::Time::now(), "base_footprint", "ultraMB"));
    loop_rate.sleep();
}

return 0;

```



```
};
```

(2) Launch 文件配置逻辑

打开工程文件夹，从文件夹 `src\sebot_robot\launch` 中找到静态坐标变换的 `launch` 相关配置文件。

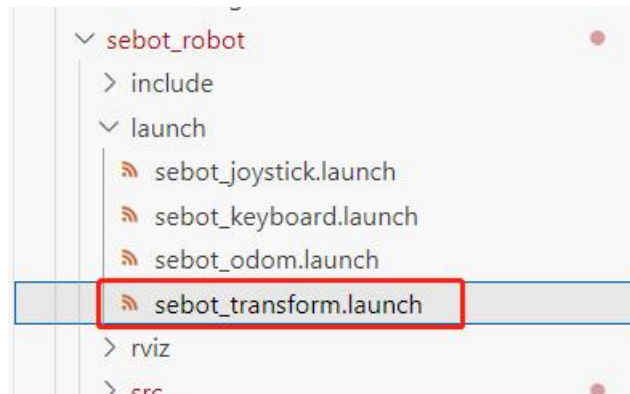


图 2-2 静态坐标配置文件

此文件是通过 `launch` 文件配置并且调用 TF 功能包实现的静态坐标变换。

文件中的主要参数含义为：

`<launch>`：该标签是任何 `roslaunch` 文件的根元素，其唯一目的是充当其他元素的容器。

`<node>`：最常见的标签，用于启动和关闭节点；

`pkg`：节点功能包名；`type`：节点类型；`name`：节点名；`args`：相关参数；`respawn`：是否自动重新启动该节点。

相关参数中一共有九个参数，按顺序分别是：`x` 轴偏移量（单位：米）、`y` 轴偏移量、`z` 轴偏移量（单位：弧度）、`z` 轴偏航角度、`y` 轴俯仰角度、`x` 轴翻滚角度、父级坐标系、子级坐标系、发布频率（单位：赫兹）

文件中每个节点参数配置均为以上九个参数，详细内容如下：

```
<launch>

  <!-- 机器人陀螺仪坐标系和底盘坐标系的位置关系 -->
  <node pkg="tf" type="static_transform_publisher"
name="base_to_imu"      args="0 0 0 0 0
0                        base_footprint    imu          30" respawn="true"/>

  <!-- 机器人激光雷达坐标系和底盘坐标系的位置关系 -->
  <node pkg="tf" type="static_transform_publisher"
name="base_to_laser"   args="0 0 0.15 -3.1416 0
0                      base_footprint    laser          30" respawn="true"/>

  <!-- 机器人深度相机坐标系和底盘坐标系的位置关系 -->
```

```

    <node pkg="tf" type="static_transform_publisher"
name="base_to_camera" args="0.05 0 1.0783 0 0
0      base_footprint      camera 30" respawn="true"/>

    <!-- 机器人超声波和底盘坐标系的位置关系 -->
    <node pkg="tf" type="static_transform_publisher"
name="base_to_ultraLF" args="0.2 0.1 0.07 0 0
0      base_footprint      ultraLF 20" respawn="true"/>
    <node pkg="tf" type="static_transform_publisher"
name="base_to_ultraMF" args="0.2 0 0.07 0 0
0      base_footprint      ultraMF 20" respawn="true"/>
    <node pkg="tf" type="static_transform_publisher"
name="base_to_ultraRF" args="0.2 -0.1 0.07 0 0
0      base_footprint      ultraRF 20" respawn="true"/>
    <node pkg="tf" type="static_transform_publisher"
name="base_to_ultraMB" args="-0.2 0 0.07 -3.1416 0
0      base_footprint      ultraMB 20" respawn="true"/>

</launch>

```

步骤 3: TF 静态坐标发布与订阅实操

(1) 查看静态坐标

在终端中复制 `sebot_transform.launch` 文件并重命名。

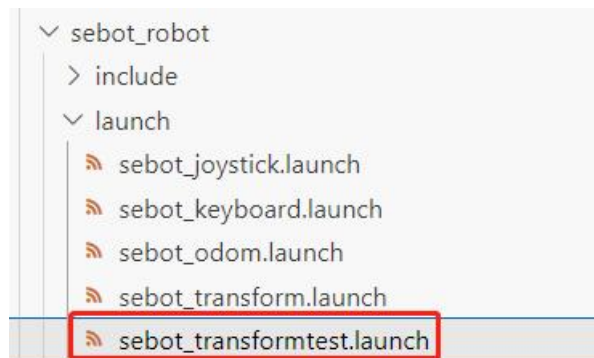


图 2-3 复制并重命名静态坐标变换的配置文件

打开重命名的文件，在`<launch>`标签底部添加以下配置并保存，此步骤是为了启动 RViz 监视器从而可视化坐标系。

```

<!-- 启动 rviz 监视机器人 -->
<node pkg = "rviz" type = "rviz" name = "rviz" args = "-d $(find
sebot_robot)/rviz/tf.rviz"/>

```

```

src > sebot_robot > launch > sebot_transformtest.launch
1 <!-- 非终版, 根据商服2.0结构进行更改 -->
2
3 <!-- 机器人陀螺仪坐标系和底盘坐标系的位置关系 -->
4 <node pkg="tf" type="static_transform_publisher" name="base_to_imu" args="0 0 0 0 0 0
5
6 <!-- 机器人激光雷达坐标系和底盘坐标系的位置关系 -->
7 <node pkg="tf" type="static_transform_publisher" name="base_to_laser" args="0 0 0.15 -3.1416 0 0
8
9 <!-- 机器人深度相机坐标系和底盘坐标系的位置关系 -->
10 <node pkg="tf" type="static_transform_publisher" name="base_to_camera" args="0.05 0 1.0783 0 0 0
11
12 <!-- 机器人超声波和底盘坐标系的位置关系 -->
13 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraLF" args="0.2 0.1 0.07 0 0 0
14 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraMF" args="0.2 0 0.07 0 0 0
15 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraRF" args="0.2 -0.1 0.07 0 0 0
16 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraMB" args="-0.2 0 0.07 -3.1416
17
18 <!-- 启动rviz监视机器人 -->
19 <node pkg="rviz" type="rviz" name="rviz" args="-d $(find sebot_robot)/rviz/tf.rviz"/>
20 </launch>

```

图 2-4 添加启动 RViz 的指令

在 sebot_robot/rviz 路径下只有一个 odom.rviz 文件，如果要启动 rviz 监视器，则需要同路径下重新复制粘贴该文件，并且将其重新命名为 tf.rviz 文件。

在终端中输入以下命令启动刚刚修改的文件(注意文件名一致)查看当前的静态坐标系。

```
roslaunch sebot_robot sebot_transformtest.launch
```

程序运行后会出现 RViz 监视器(一种可视化工具)，监视器内会出现我们配置好的静态坐标。其中红色为 x 轴，绿色为 y 轴，蓝色为 z 轴。通过鼠标可操控 RViz 右侧建模界面，鼠标左键旋转视角，上/下滚动滚轮可放大/缩小视角，按住滚轮移动鼠标可平移视角。如下图所示，调整视角后可查看清晰的坐标显示。

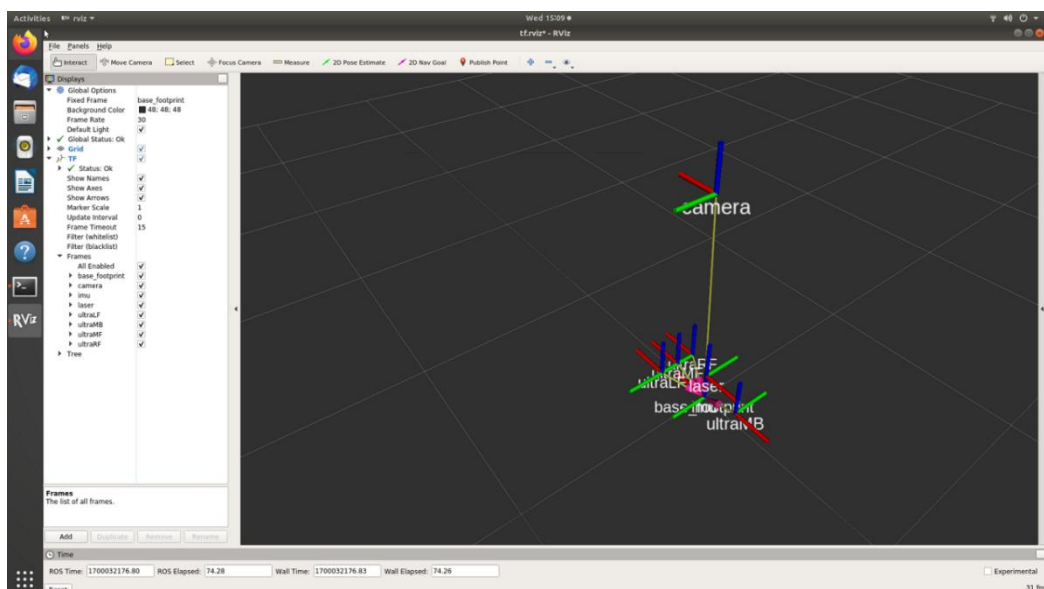


图 2-5 查看静态坐标

左侧的菜单选项可以对坐标系进行调整。我们需要使用的是 TF 功能，如果没有 TF 功

能，则点击下方“Add”，在弹出界面中选择“TF”。

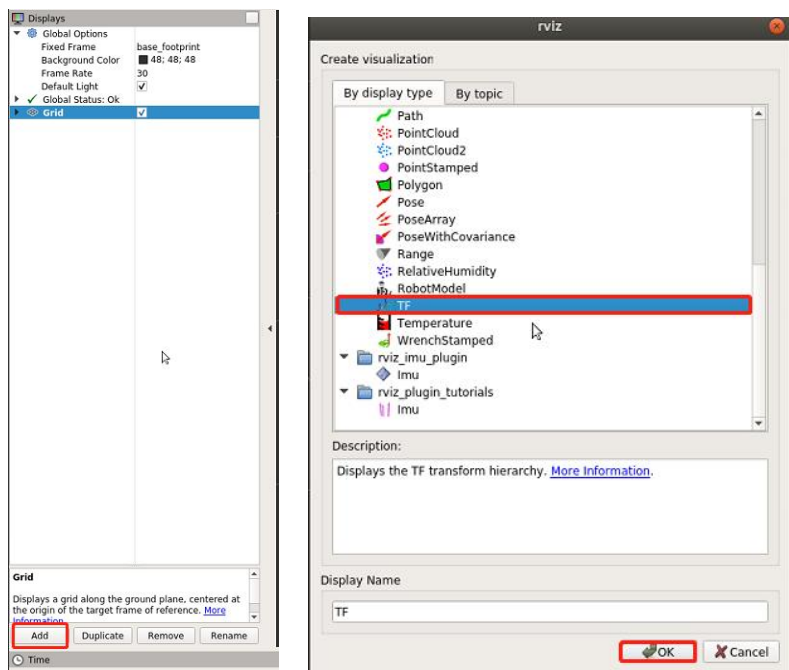


图 2-6 添加 TF 选项

将 TF 添加至左侧的菜单栏后，其下的选项“Frames”下即我们创建好的静态坐标系，包括底盘坐标、相机坐标、里程计、激光雷达等坐标系，打勾表示显示坐标系。上方选项则是相关设置，可以根据需要自行测试和调整，需要注意的是“Fixed Frame”选项，此选项表示固定参考系，后面的参数即设置了具体的坐标系名称，其他坐标系的查看均建立在此固定坐标系上，查看坐标系时如果出现显示问题，注意调整此选项。后续的相关操作也是同理。

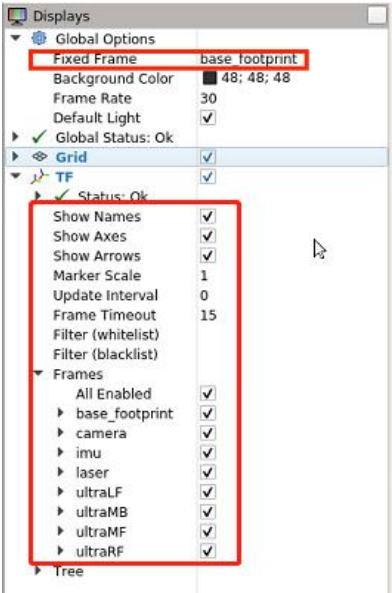


图 2-7 设置 TF 选项

如果需要保存设置或图像则点击左上方“File”选项即可。

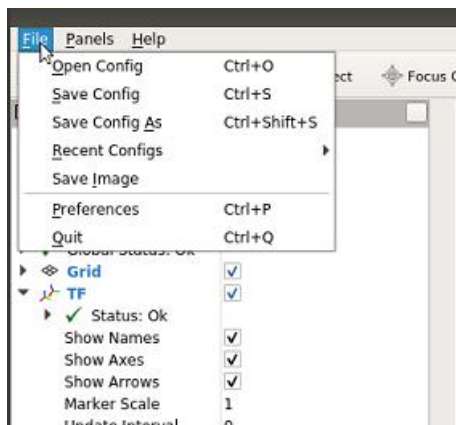


图 2-8 保存按钮

(2) 添加静态坐标

下面我们在 launch 文件中添加一个新的静态坐标，添加时需要正确设置相关参数。

在 `sebot_transformtest.launch` 文件中根据已有静态坐标的写法，添加以下配置，然后重新启动此 launch 文件。

```
<node pkg="tf" type="static_transform_publisher" name="base_to_DIY"
args="0.5 0 0.5 -3.1416 0 0 base_footprint DIY 20" respawn="true"/>

src > sebot_robot > launch > sebot_transformtest.launch
1 launch> <!-- 非终版，根据商服2.0结构进行更改 -->
2
3 <!-- 机器人陀螺仪坐标系和底盘坐标系的位置关系 -->
4 <node pkg="tf" type="static_transform_publisher" name="base_to_imu" args="0 0 0 0 0 base_footprint imu 30"
5
6 <!-- 机器人激光雷达坐标系和底盘坐标系的位置关系 -->
7 <node pkg="tf" type="static_transform_publisher" name="base_to_laser" args="0 0 0.15 -3.1416 0 0 base_footprint laser 30"
8
9 <!-- 机器人深度相机坐标系和底盘坐标系的位置关系 -->
10 <node pkg="tf" type="static_transform_publisher" name="base_to_camera" args="0.05 0 1.0783 0 0 0 base_footprint camera 30" res
11
12 <!-- 机器人超声波和底盘坐标系的位置关系 -->
13 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraF" args="0.2 0.1 0.07 0 0 0 base_footprint ultraF 20"
14 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraMF" args="0.2 0 0.07 0 0 0 base_footprint ultraMF 20"
15 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraRF" args="0.2 -0.1 0.07 0 0 0 base_footprint ultraRF 20"
16 <node pkg="tf" type="static_transform_publisher" name="base_to_ultraMB" args="-0.2 0 0.07 -3.1416 0 0 base_footprint ultraMB 20"
17 <node pkg="tf" type="static_transform_publisher" name="base_to_DIY" args="0.5 0 0.5 -3.1416 0 0 base_footprint DIY 20" respawn="true"/>
18 <!-- 启动rviz监视机器人 -->
19 <node pkg="rviz" type="rviz" name="rviz" args="-d $(find sebot_robot)/rviz/tf.rviz"/>
20 launch>
21
```

图 2-9 添加静态坐标

可以看到 RViz 监视器中出现了我们自己发布的 DIY 坐标系。实操时也可以改变相关参数的数值，然后查看改变数值后坐标系的变化。

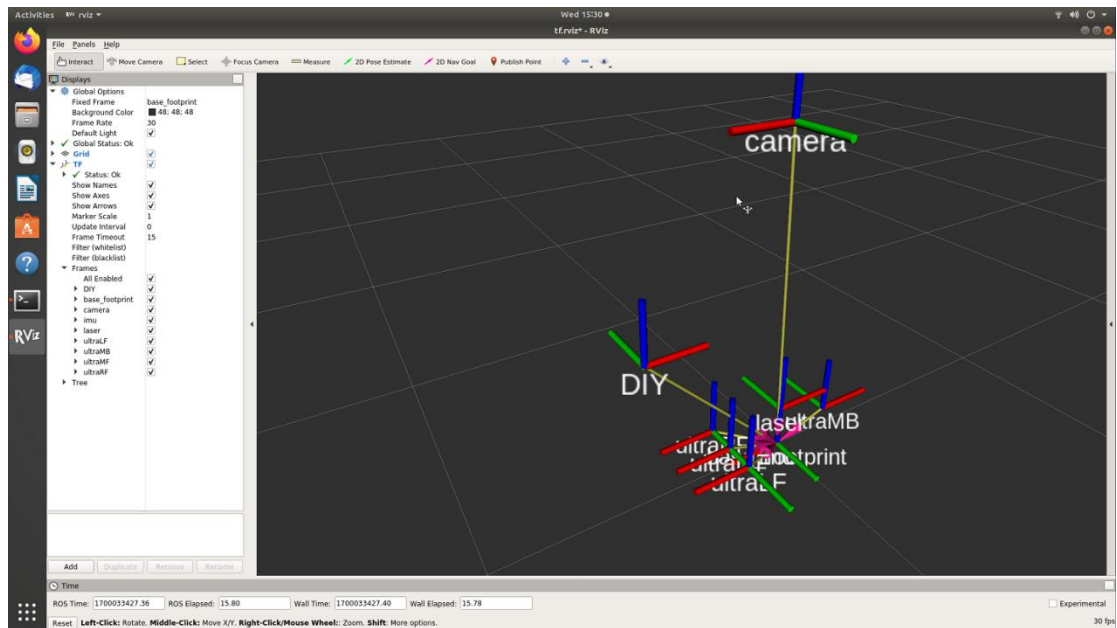


图 2-10 查看添加后的静态坐标

(3) 查看静态坐标话题及 TF 树

打开新终端，输入以下命令查看话题列表。

```
rostopic list
```

```
root@paddlepi:~# rostopic list
/clicked_point
/initialpose
/move_base_simple/goal
/rosout
/rosout_agg
/tf
/tf_static
root@paddlepi:~#
```

图 2-11 查看话题列表

可以看到如上图的话题列表中有/tf、/tf_static 等通信话题，然后输入以下命令查看有关 tf 的话题消息。运行后终端内消息不断刷新，即 launch 文件配置和发布的静态坐标消息。

```
rostopic echo /tf
```



```
root@paddlepi: ~
File Edit View Search Terminal Help

rotation:
  x: 0.0
  y: 0.0
  z: 0.0
  w: 1.0
---
transforms:
-
  header:
    seq: 0
    stamp:
      secs: 1700033792
      nsecs: 746196839
    frame_id: "base_footprint"
  child_frame_id: "ultraRF"
  transform:
    translation:
      x: 0.2
      y: -0.1
      z: 0.07
    rotation:
      x: 0.0
      y: 0.0
      z: 0.0
```

图 2-12 查看静态坐标话题消息

打开新终端，输入以下命令即可查看 TF 树，TF 树即坐标系之间关系的图形化展现。

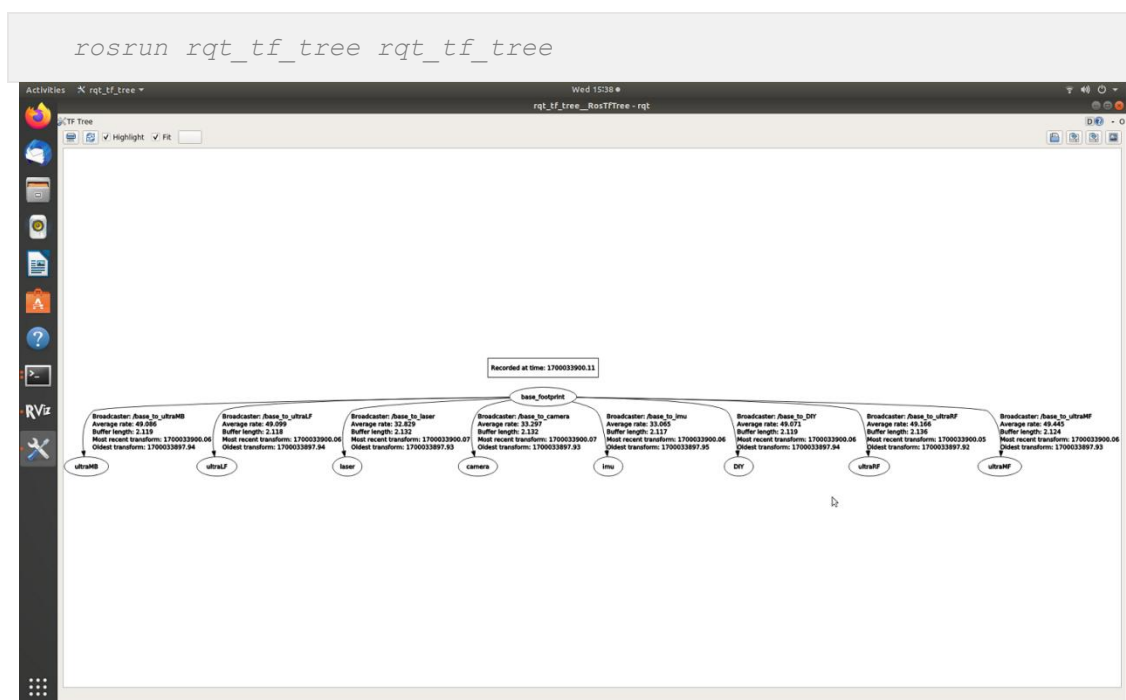


图 2-13 查看静态坐标 TF 树

在自定义的工作空间内进行此步骤时，可直接创建对应的 launch 文件夹以及内部的 launch 文件，然后使用 roslaunch 命令执行即可。

步骤 4：TF 动态坐标发布逻辑

动态坐标系在 ROS 机器人使用频率很高，因此经常会被调用，其逻辑也比较复杂，往往需要不同的源码来发布和接收相关话题的内容，同时还需要相关的配置文件来进行参数配

置和节点调用等。下面我们以商服机器人的移动控制程序为例，理解其节点功能内的动态坐标转换逻辑。

在商服机器人内部，应用了里程计进行定位判断，里程计是一种利用从移动传感器获得的数据来估计物体位置随时间变化而改变的方法。在 ROS 中，里程计信息可以分为两个部分：一个是位姿（位置和姿态），一个是速度（线速度和角速度）。里程计可以利用惯性传感器（IMU）、轮式编码器等传感器的数据。对于 ROS 系统来说，里程计是一个模块，而不是一种具体的传感器。它可以从安装在发动机和轮子上的编码器获取数据，也可以从移动传感器中获得数据。通过对里程计中的数据进行融合，可以得到更准确的定位数据。

在我们学习的商服机器人中，主要是通过编码器和 IMU 来进行融合计算。在融合过程中，可以使用一些算法（如卡尔曼滤波器）来处理这些数据，以进一步优化定位结果。编码器融合里程计可以提供更准确、更可靠的定位数据，从而有助于实现更精确的路径规划和导航。其逻辑算法复杂所以我们不做深入分析，理解其大致的计算和处理逻辑即可。想要深入理解其原理可自行查看工程文件和相关资料。

（1）源码逻辑

打开工程文件夹，从文件夹 `src\sebot_robot\src` 中找到机器人控制的相关源码。

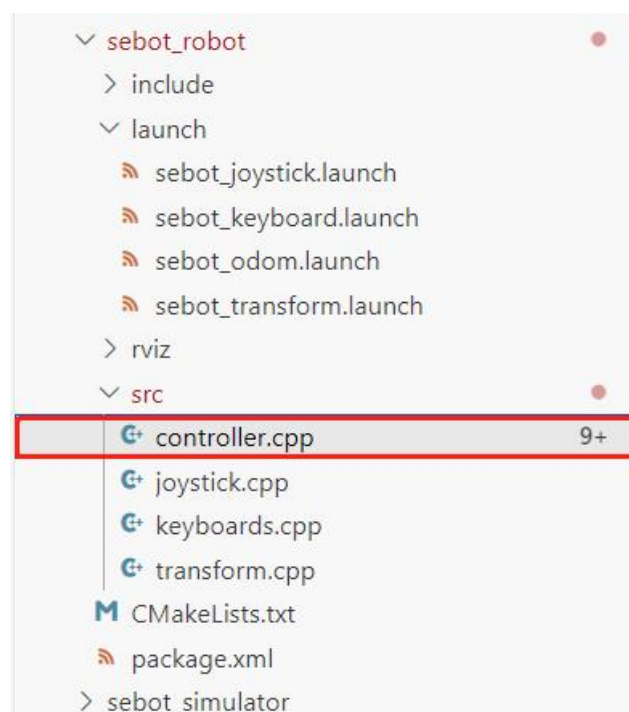


图 2-14 机器人控制源码

此程序是商服机器人总体的控制功能，在后续的建图和导航等功能中均会用到此程序，

这里我们主要学习其中关于动态坐标部分的代码逻辑。

此文件中，与动态坐标变换相关的函数为 `topicPublishOdom()` 和 `topicPublishImu()`，下面分别讲解其程序逻辑。

`topicPublishOdom()` 函数主要是用于进行机器人的位置计算，通过将编码器与里程计的数据进行融合，以提高定位精度。

在程序逻辑中，会根据机器人是否发生真实运动来选择采用里程计的数据还是编码器的数据，机器人在没有控制信号（即手柄或键盘控制）的情况下发生运动时，以编码器采集的数据为准；通过控制信号发生运动时则以里程计的数据为准。在这种控制方式下，无论机器人是否发生实际运动都可以获取较为准确的数据。

函数最后会发布相关的话题消息数据（`pubOdom.publish(msgOdom)`），此数据发出后可由相关功能进行订阅，从而用于其他功能的实现。

```
/**
    pubImu.publish(msgImu); // 发布 imu 话题数据
}
/**
 * @brief 里程计数据话题发布
 */
void topicPublishOdom(void)
{
    nav_msgs::Odometry msgOdom; // 定义里程计 (odom) 消息类型
    geometry_msgs::Quaternion rotation; // 定义四元数变量
    rotation = tf::createQuaternionMsgFromYaw(uart->odomStr.angleRad);
    // odom 的偏航角需要转换成四元数才能发布

    //[TF] odom 坐标系转换
    if (!imuEnable) // 启动 IMU 融合里程计后不发布编码器 odom 坐标系
    {
        geometry_msgs::TransformStamped tfOdom; // 定义 (tf) 消息类型
        tfOdom.header.stamp = ros::Time::now(); // 载入 tf 时间戳
        tfOdom.header.frame_id = "odom"; // 定义 TF 父子关系
        tfOdom.child_frame_id = "base_footprint"; // 定义 TF 父子关系
        tfOdom.transform.translation.x = uart->odomStr.moveX; // tf 关于 x 轴的数据
        tfOdom.transform.translation.y = uart->odomStr.moveY; // tf 关于 y 轴的数据
        tfOdom.transform.translation.z = 0.0; // tf 关于 z 轴的数据
        tfOdom.transform.rotation = rotation; // tf 航向参数
        static tf::TransformBroadcaster odomBroadcaster; // 定义 tf 发布者
```

```

        odomBroadcaster.sendTransform(tfOdom); // 发布从 odom 到
base_footprint 的坐标变换
    }

    //[Topic] odom 话题数据
    msgOdom.header.stamp = ros::Time::now(); // 载入 odom 时间戳
    msgOdom.header.frame_id = "odom"; // 定义 TF 父子关系
    msgOdom.child_frame_id = "base_footprint"; // 定义 TF 父子关系
    msgOdom.pose.pose.position.x = uart->odomStr.moveX; // odom 关于 x 轴
的数据
    msgOdom.pose.pose.position.y = uart->odomStr.moveY; // odom 关于 y 轴
的数据
    msgOdom.pose.pose.position.z = 0.0; // odom 关于 z 轴的数据
    msgOdom.pose.pose.orientation = rotation; // odom 航向角
    msgOdom.twist.twist.linear.x = uart->odomStr.speedLineX; // x 轴线
速度
    // msgOdom.twist.twist.linear.y = uart->odomStr.speedLineY; // y
轴线速度
    msgOdom.twist.twist.angular.z = uart->odomStr.speedAngle; // odom
角速度

    // 里程计协方差矩阵:
    // 如果机器人未运动, 说明编码器的误差会比较小, 认为编码器数据更可靠
    if (msgOdom.twist.twist.linear.x == 0 &&
msgOdom.twist.twist.linear.y == 0)
    {
        msgOdom.pose.covariance = {1e-9, 0, 0, 0, 0, 0,
                                0, 1e-3, 1e-9, 0, 0, 0,
                                0, 0, 1e6, 0, 0, 0,
                                0, 0, 0, 1e6, 0, 0,
                                0, 0, 0, 0, 1e6, 0,
                                0, 0, 0, 0, 0, 1e-9};
        msgOdom.twist.covariance = {1e-9, 0, 0, 0, 0, 0,
                                0, 1e-3, 1e-9, 0, 0, 0,
                                0, 0, 1e6, 0, 0, 0,
                                0, 0, 0, 1e6, 0, 0,
                                0, 0, 0, 0, 1e6, 0,
                                0, 0, 0, 0, 0, 1e-9};
    }
    else // 考虑到运动中编码器可能带来的滑动误差, 选择 imu 的数据更可靠
    {
        msgOdom.pose.covariance = {1e-3, 0, 0, 0, 0, 0,
                                0, 1e-3, 0, 0, 0, 0,
                                0, 0, 1e6, 0, 0, 0,

```

```

0, 0, 0, 1e6, 0, 0,
0, 0, 0, 0, 1e6, 0,
0, 0, 0, 0, 0, 1e3};
msgOdom.twist.covariance = {1e-3, 0, 0, 0, 0, 0,
0, 1e-3, 0, 0, 0, 0,
0, 0, 1e6, 0, 0, 0,
0, 0, 0, 1e6, 0, 0,
0, 0, 0, 0, 1e6, 0,
0, 0, 0, 0, 0, 1e3};
}
pubOdom.publish(msgOdom); // 发布 odom 话题数据
}

```

topicPublishImu () 函数相对来说较为简单，主要是读取惯性传感器 IMU 传感器的相关数据，以此判断机器人当前的姿态（即朝向），IMU 传感器比较精准，因此不需要使用特殊的技术手段来提高精度，可以直接读取数据并发布话题消息。

```

/**
 * @brief Imu 里程计数据话题发布
 */
void topicPublishImu(void)
{
    sensor_msgs::Imu msgImu; // 定义 (imu) 消息类型

    msgImu.header.stamp = ros::Time::now(); // 载入 imu 时间戳
    msgImu.header.frame_id = "imu"; // imu 坐标系名称

    // 线加速度
    msgImu.linear_acceleration.x = uart->imuStr.accx; // x 轴线加速度
    msgImu.linear_acceleration.y = uart->imuStr.accy; // y 轴线加速度
    msgImu.linear_acceleration.z = uart->imuStr.accz; // z 轴线加速度
    // 角速度
    msgImu.angular_velocity.x = uart->imuStr.gyrox; // x 轴角速度
    msgImu.angular_velocity.y = uart->imuStr.gyroy; // y 轴角速度
    msgImu.angular_velocity.z = uart->imuStr.gyroz; // z 轴角速度
    msgImu.angular_velocity_covariance[0] = 1e6; // 三轴角速度协方差矩阵
    msgImu.angular_velocity_covariance[4] = 1e6;
    msgImu.angular_velocity_covariance[8] = 1e-6;
    // 四元素解算
    geometry_msgs::Quaternion orientation =
        tf::createQuaternionMsgFromYaw(uart->imuStr.yaw); // 只参考航向
    角解算四元素
    msgImu.orientation = orientation;
    msgImu.orientation_covariance[0] = 1e6;
    msgImu.orientation_covariance[4] = 1e6;
}

```

```

msgImu.orientation_covariance[8] = 1e-6;

pubImu.publish(msgImu); // 发布 imu 话题数据
}

```

其中，`pubImu.publish(msgImu)`即发布 imu 话题数据。

（2）配置逻辑

打开工程文件夹，从文件夹 `src\sebot_robot\launch` 中找到动态坐标变换的相关配置文件。

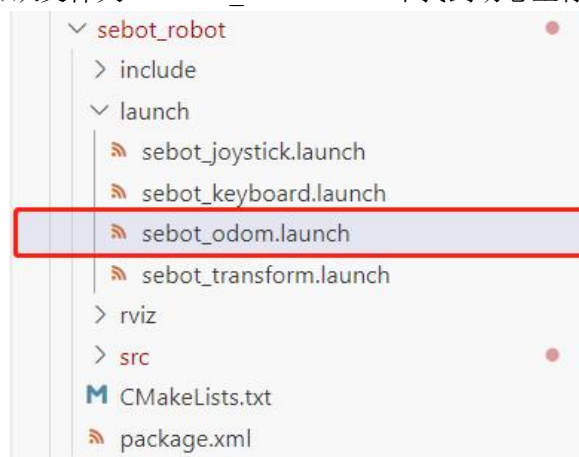


图 2-15 里程计定位配置文件

相比于前面的静态坐标配置文件，此配置文件中多了一些相关元素：`<param>`标签定义要在参数服务器上设置的参数，包括其名称 `name`、类型 `type`、数值 `value`，可以将其简单理解为将具体的参数传入到指定的源码中。此处设置的三个`<param>`标签即传入到 `controller.cpp` 文件中的参数，以此开启对应的使能，使能开启才能执行源码中对应的功能。另外一个`<include>`标签，该标签可以将另一个配置文件导入当前文件。其他部分的原理与静态坐标配置文件相同。

```

<launch>
  <!-- 启动机器人的控制节点 -->
  <node pkg="sebot_robot" type="sebot_controller"
name="sebot_controller" respawn="true">
    <!-- 是否发布传感器话题 value: true 发布 false 不发布 -->
    <param name="ultraEnable" type="bool"
value="false"/>    <!-- 超声波使能 -->
    <param name="odomEnable" type="bool"
value="true"/>    <!-- 里程计使能 -->
    <param name="imuEnable" type="bool" value="true"/> <!-- imu 使
能 -->
  </node>

  <!-- 机器人陀螺仪坐标系和底盘坐标系的位置关系 -->

```

```

<node pkg="tf" type="static_transform_publisher" name="base_to_imu"
args="0 0 0 0 0 0 base_footprint imu 30" respawn="true"/>

<!-- 启动融合里程计 -->
<include file = "$(find
robot_pose_ekf)/launch/robot_pose_ekf.launch"/>

<!-- 启动遥控手柄控制结点 -->
<node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick"
respawn="true"/>

<!-- 启动 rviz 监视机器人 -->
<node pkg = "rviz" type = "rviz" name = "rviz" args = "-d $(find
sebot_robot)/rviz/odom.rviz"/>
</launch>

```

步骤 5: TF 动态坐标发布与订阅实操

(1) 查看动态坐标

在终端中输入以下命令查看当前的里程计动态坐标系。

```
roslaunch sebot_robot sebot_odom.launch
```

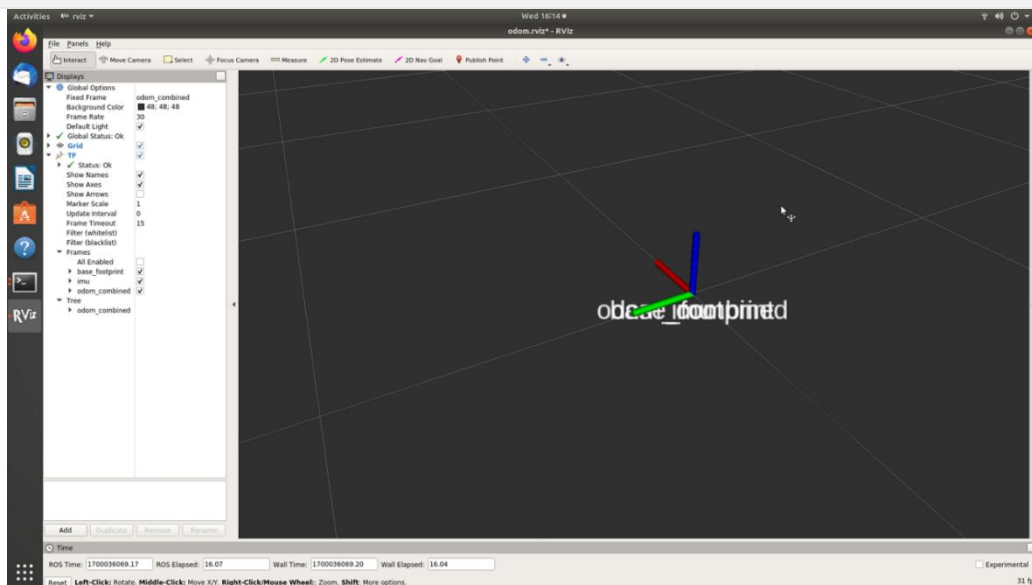


图 2-16 查看动态坐标

程序执行后可以看到三个叠加在一起的坐标系，通过手柄控制机器人移动和旋转后，即可看到坐标系分开，其中里程计坐标系是不发生改变的，而 IMU 坐标系和底盘坐标系相对静止，因此会一起移动。如果需要设置则在左边的菜单选项中操作即可。

话题的发布与接收操作

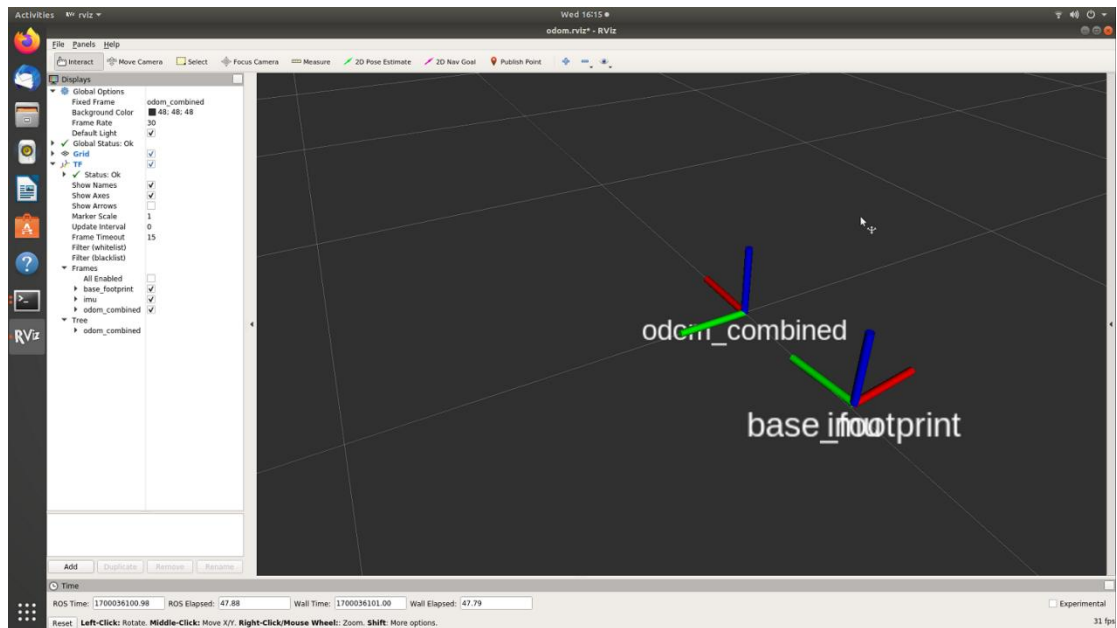


图 2-17 查看动态坐标（变化后）

（2）查看动态坐标话题消息及 TF 树

打开新终端，输入以下命令查看话题列表。

```
rostopic list
```

```
root@paddlepi:~# rostopic list
/clicked_point
/cmd_vel
/imu
/initialpose
/move_base_simple/goal
/odom
/odom_combined
/rosout
/rosout_agg
/tf
/tf_static
root@paddlepi:~#
```

图 2-18 查看话题列表

然后输入以下命令查看话题消息。与动态坐标相关的话题有三个，分别是/*imu*、/*odom*、以及/*odom_combined*。任选其中的话题执行输出消息命令后，终端内消息不断刷新，即发送的动态坐标相关的话题消息数据。

```
rostopic echo 话题名
```



```
root@paddlepi: ~  
File Edit View Search Terminal Help  
stamp:  
  secs: 1700038051  
  nsecs: 765230536  
  frame_id: "imu"  
orientation:  
  x: 0.0  
  y: 0.0  
  z: -0.394705818977  
  w: 0.918807551376  
orientation_covariance: [1000000.0, 0.0, 0.0, 0.0, 1000000.0, 0.0, 0.0, 0.0, 1e-06]  
angular_velocity:  
  x: 0.0  
  y: 0.0  
  z: 0.0  
angular_velocity_covariance: [1000000.0, 0.0, 0.0, 0.0, 1000000.0, 0.0, 0.0, 0.0, 1e-06]  
linear_acceleration:  
  x: 0.0  
  y: 0.0  
  z: -9.0  
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]  
---
```

图 2-19 查看 imu 话题消息内容

打开新终端，输入以下命令即可查看 TF 树，TF 树即坐标系之间关系的图形化展现。

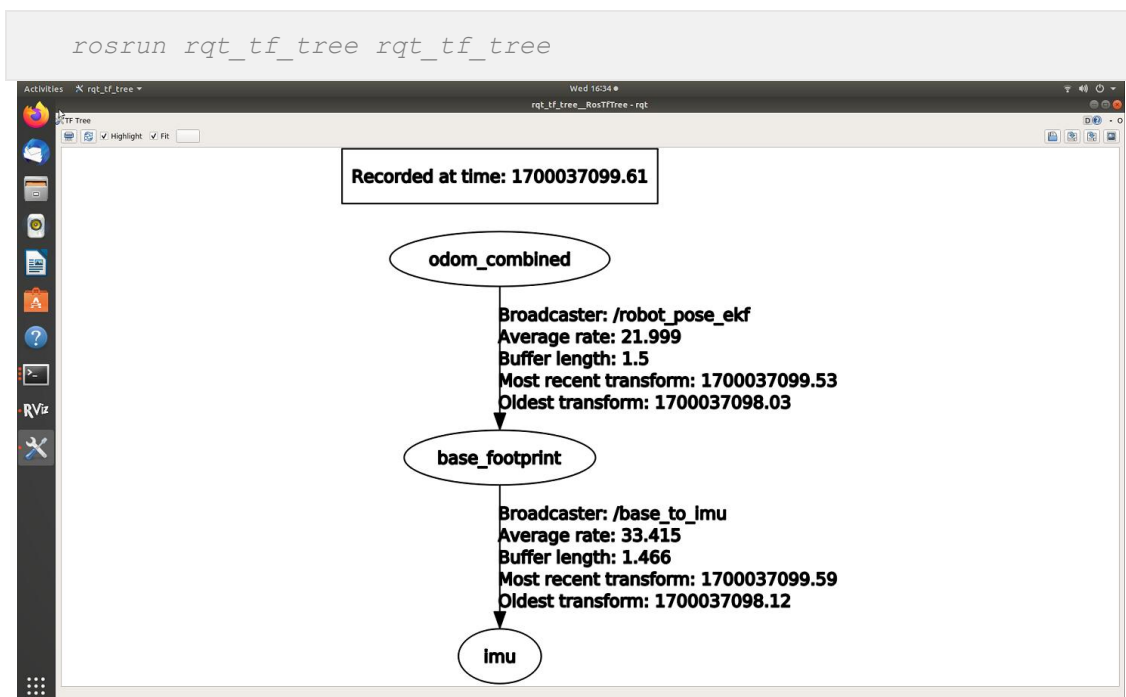


图 2-20 查看动态坐标 TF 树